D Y PATIL UNIVERSITY
NAVI MUMBAI

Subject Name : Machine Learning

Unit No: 2

Unit Name : Supervised Learning: Regression and Classification

Faculty Name : Dr. Ritu Jain

D Y PATIL UNIVERSITY
NAVI MUMBAI

Unit No.: 2.2

Faculty Name : Dr Ritu Jain

Simple Linear Regression: Cost Function

Now, we will see the original hypothesis and corresponding cost function.

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

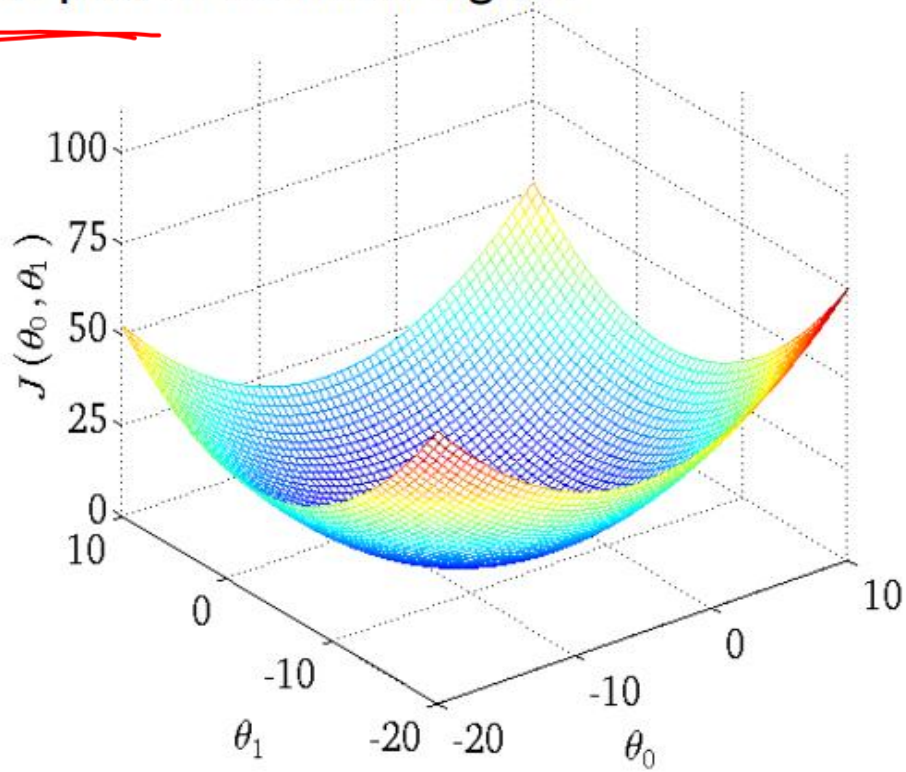
Simple Linear Regression: Cost Function

- Using our original complex hypothesis with two variables,
  - So cost function is
    - $J(\theta_0, \theta_1)$
- Example,
  - Say
    - $\theta_0 = 50$
    - $\theta_1 = 0.06$
  - Previously we plotted our cost function by plotting
    - θ_1 vs $J(\theta_1)$
  - Now we have two parameters
    - Plot becomes a bit more complicated
    - **Generates a 3D surface plot where axis are**
      - $X = \theta_1$
      - $Z = \theta_0$
      - $Y = J(\theta_0,\theta_1)$

Dr. Ritu Jain

Simple Linear Regression: Cost Function

Contour plot or Contour figure



Gradient Descent

- Finding the value of θ_0,θ_1 manually which minimizes the cost function J is very difficult.
  - What we really want is an efficient algorithm for finding the minimum for θ_0 and θ_1
- **Gradient Descent Algorithm** is used for finding θ_0,θ_1 which minimize cost function J.
- It is an Iterative algorithm
- Gradient descent is used all over machine learning for minimization
- Start by looking at a general J() function
- We will start with a general problem
  - We have $J(\theta_0, \theta_1)$
  - We want to get **min $J(\theta_0, \theta_1)$**
- Gradient descent can be applied to more general functions
  - $J(\theta_0, \theta_1, \theta_2 .... \theta_n)$
  - min $J(\theta_0, \theta_1, \theta_2 .... \theta_n)$

Dr. Ritu Jain

Gradient Descent for Simple Linear Regression

Algorithm:

1. Start with initial guesses for θs. Let's say 0,0 (or any other value)
2. Keeping changing θ_0 and θ_1 a little bit to try and reduce $J(\theta_0,\theta_1)$
3. Each time you change the parameters, you select the gradient which reduces $J(\theta_0,\theta_1)$ the most possible (steepest decrease of J).
4. Repeat 2 and 3 until you converge to a local minimum

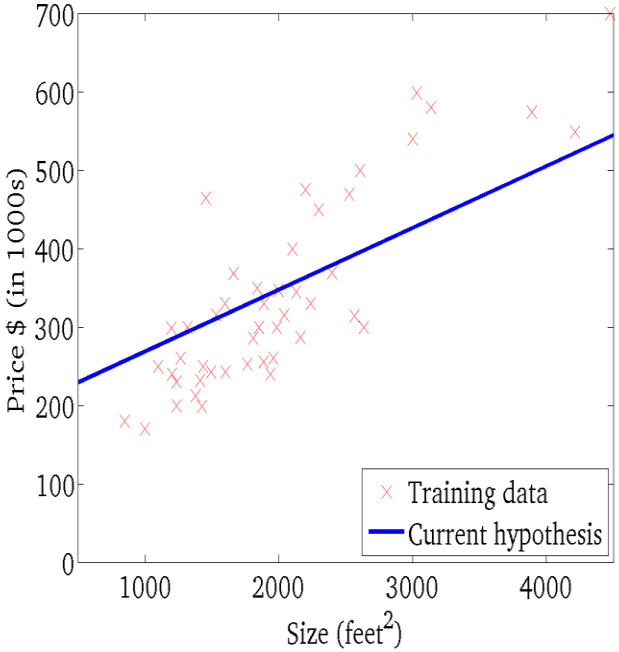
Dr. Ritu Jain

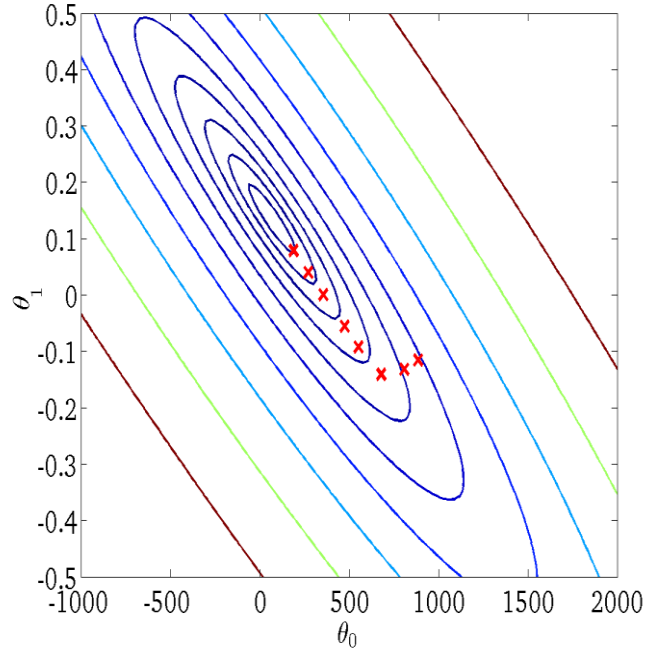
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)

$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)





Gradient Descent

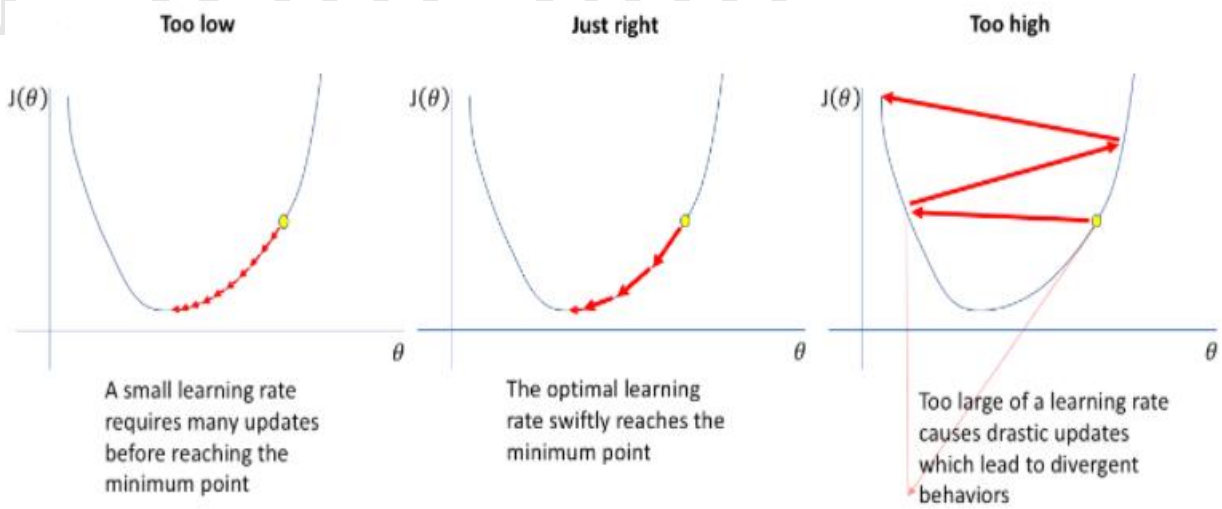
- What does this all mean?
  - Update θ_j by setting it to $(\theta_j - \alpha)$ times the partial derivative of the cost function with respect to θ_j
- α (alpha)is a number called the **learning rate**
  - Controls how big a step you take
    - If α is big, we will have an aggressive gradient descent
    - If α is small, gradient descent take tiny steps
- Lets take a simple example, consider only θ_1. So, consider $J(\theta_1)$

Dr. Ritu Jain

Learning Rate

- For simplicity, let us first consider a function of a single variable
- **Learning rate (α):** What happens if alpha is too small or too large
- Too small
  - Take baby steps
  - Takes too long
- Too large
  - Can overshoot the minimum and fail to converge

Dr. Ritu Jain



Gradient Descent

- **Batch Gradient Descent:**
  - Each step of gradient descent uses all the training examples.
  - Calculates the error for each example in the training dataset, but only updates the model after all training examples have been evaluated.
- **Stochastic Gradient Descent:**
  - In this algorithm, we repeatedly run through the training set,
  - each time we encounter a training example, we update the parameters according to the gradient of the error with respect to that single training example only.
  - calculates the error and updates the model for each example in the training dataset.

Dr. Ritu Jain

Gradient Descent

- Whereas **batch gradient descent** has to scan through the entire training set before taking a single step—a costly operation if m is large.
- **Stochastic gradient descent** can start making progress right away, and continues to make progress with each example it looks at.
- Often, **stochastic gradient descent** gets θ "close" to the minimum much faster than **batch gradient descent**. (Note however that it may never "converge" to the minimum, and the parameters θ will keep oscillating around the minimum of J(θ); but in practice most of the values near the minimum will be reasonably good approximations to the true minimum).
- For these reasons, particularly when the training set is large, stochastic gradient descent is often preferred over batch gradient descent.

Dr. Ritu Jain

Multivariate Linear Regression

Living area (feet^2)	#bedrooms	Price (1000$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
$\vdots$	$\vdots$	$\vdots$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

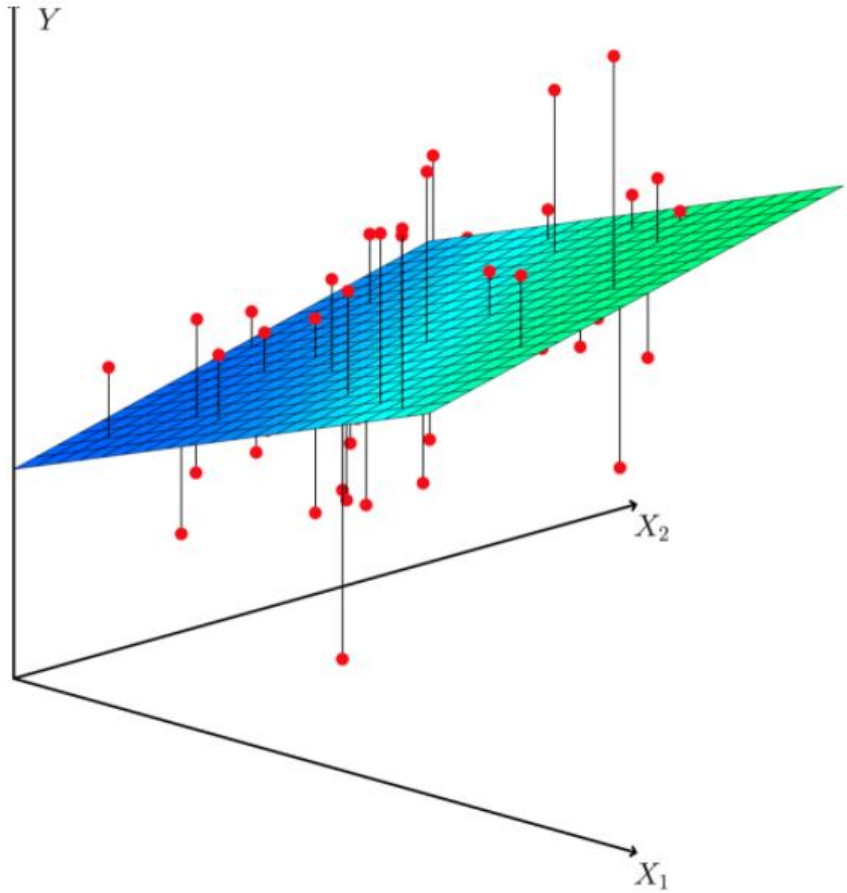


Fig. 15 Multiple linear regression

Linear Regression with multiple variables

Hypothesis: $h_\theta(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \ldots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \ldots, \theta_n) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

Gradient descent:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \ldots, \theta_n)$$

$\}$ (simultaneously update for every $j = 0, \ldots, n$)

Linear Regression with multiple variables

Gradient Descent

Previously (n=1):

Repeat $\{$

$$\theta_0 := \theta_0 - \alpha \underbrace{\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial\theta_0}J(\theta)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x^{(i)}$$

(simultaneously update θ_0, θ_1)

$\}$

New algorithm $(n \geq 1)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$$

(simultaneously update θ_j for $j = 0, \ldots, n$)

$\}$

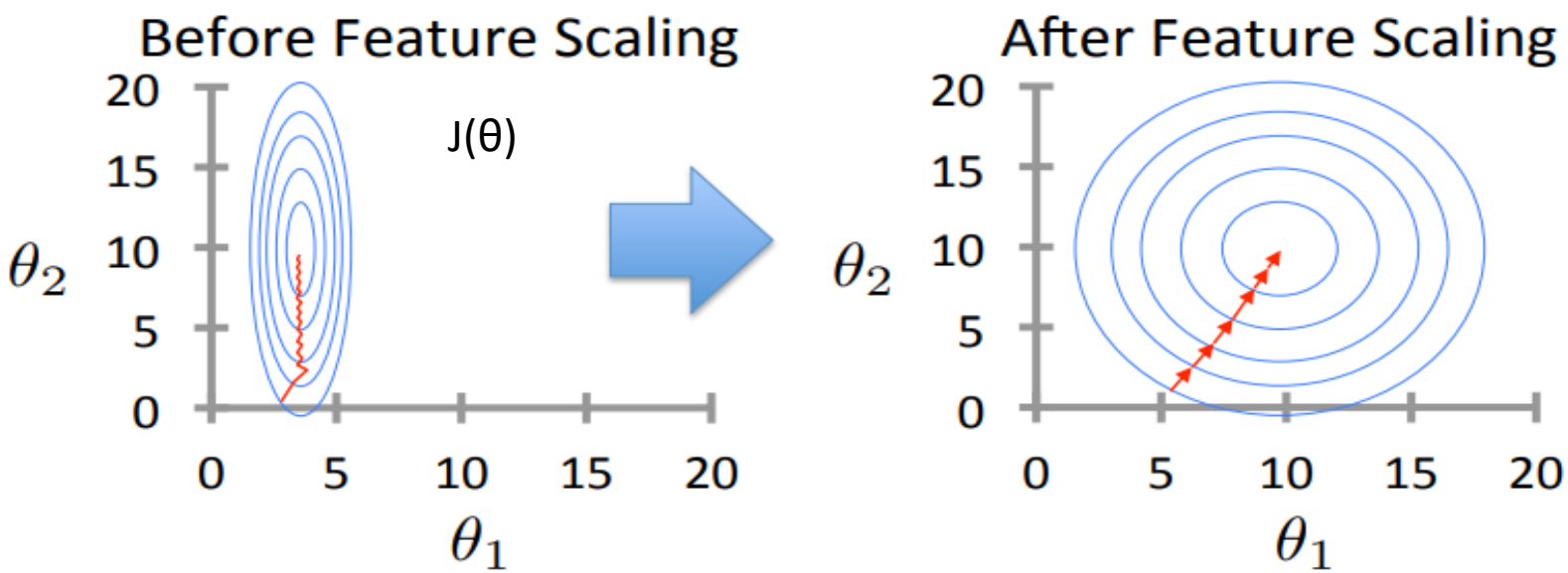
$$\theta_0 := \theta_0 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_2^{(i)}$$

$\ldots$

Effect of Feature Scaling on gradient descent



If x1 takes much larger values than x2,Countours will be very skewed and elliptical

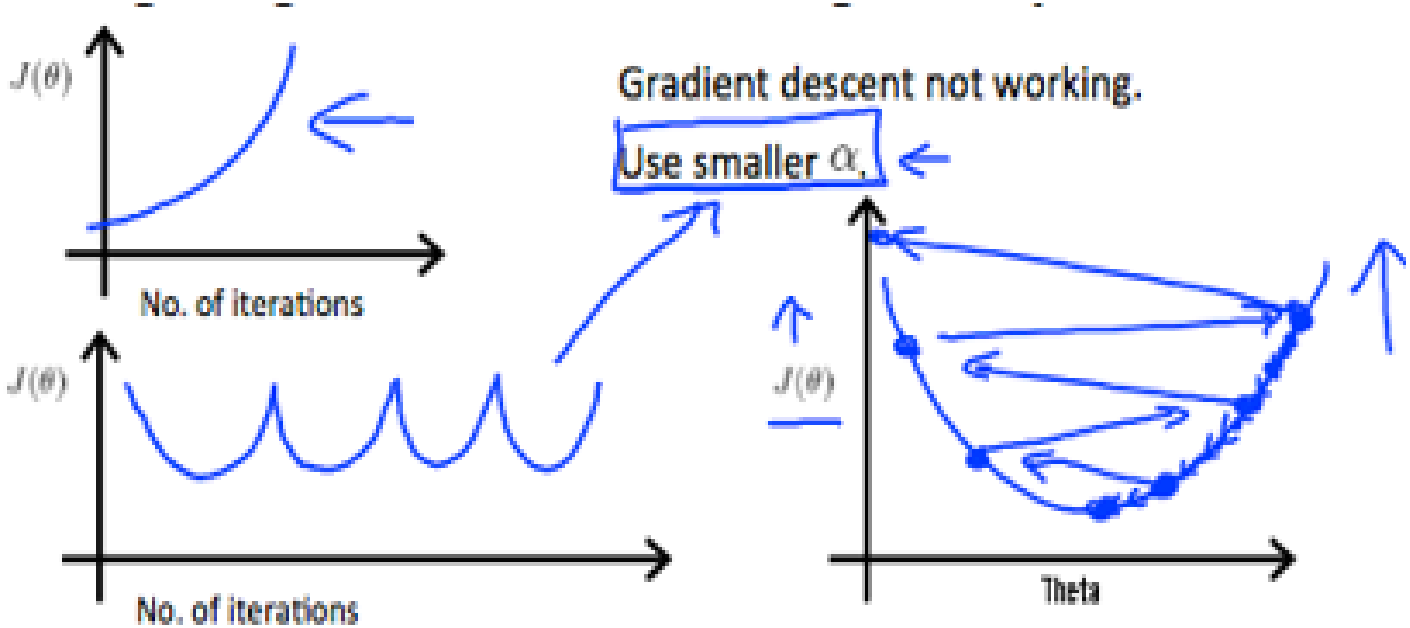
- Makes gradient descent converge *much* faster

Is gradient descent working properly?

- Plot how J(θ) changes with every iteration of gradient descent

Dr. Ritu Jain

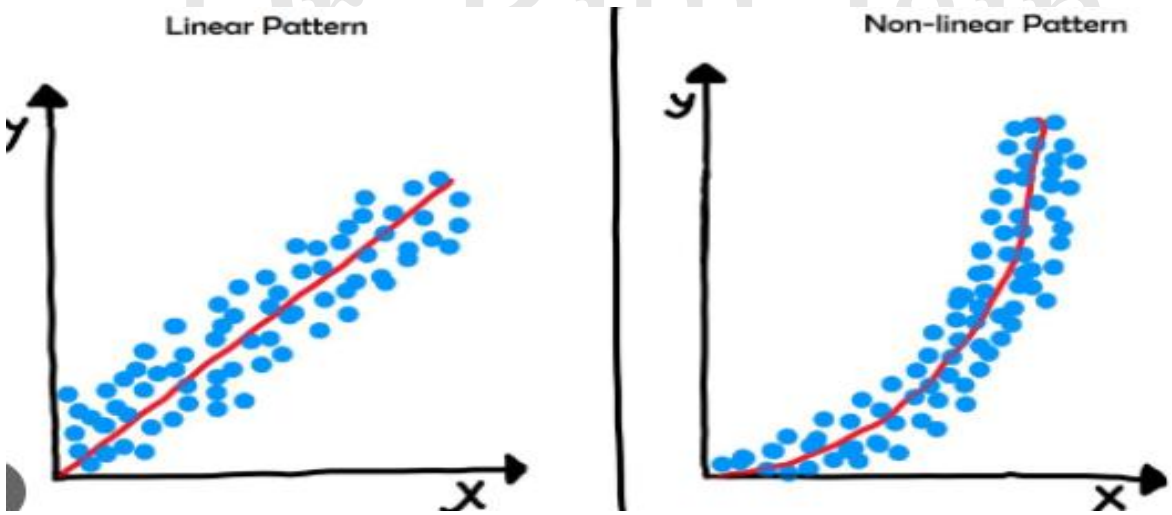
Is gradient descent working properly?



Assumptions of Linear Regression

- Linear Regression is a parametric model which needs its assumptions to be satisfied, so if we are applying linear regression. If this assumption is not satisfied, then the model built will be very poor and less accurate.
- **Assumes Linearity:** The method assumes the relationship between the variables is linear. If the relationship is non-linear, linear regression might not work well.

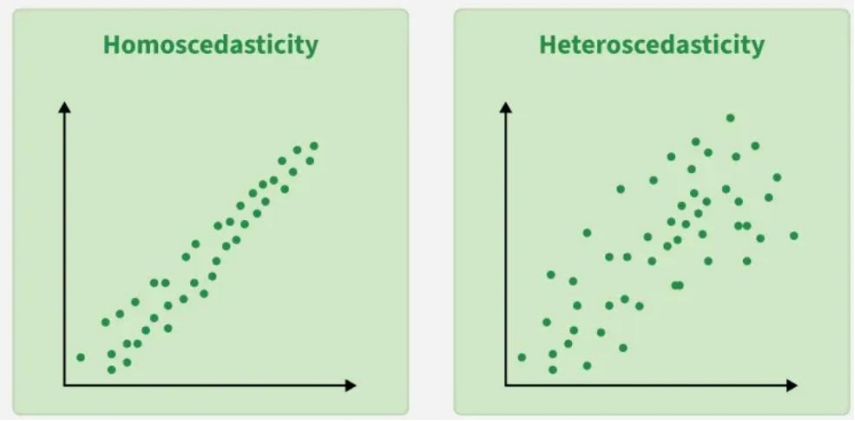
Dr. Ritu Jain



Assumptions of Linear Regression

- **Sensitivity to Outliers:** Outliers can significantly affect the slope and intercept, skewing the best-fit line.
- **Independence of Errors**: The errors in predictions should not affect each other.
- **Constant Variance (Homoscedasticity):** The errors should have equal spread across all values of the input. If the spread changes (like fans out or shrinks), it's called heteroscedasticity and it's a problem for the model.

Dr. Ritu Jain



Assumptions of Linear Regression

- **Normality of Errors**: The errors should follow a normal (bell-shaped) distribution.
- **No Multicollinearity (for multiple regression)**: there should be **less or no** multicollinearity in the given dataset. This situation occurs when the features or independent variables of a given dataset are highly correlated to each other.
- In a model having correlated variables, it becomes difficult to determine which variable is contributing to predict the target variable. Another thing is, the standard errors tend to **increase** due to the presence of correlated variables.

Dr. Ritu Jain

Evaluation Metrics for Linear Regression

- A variety of evaluation measures can be used to determine the strength of any linear regression model.
- **Evaluation metrics are a measure of how good a model performs and how well it approximates the relationship.**
- The most common measurements are:
  - **Mean Square Error (MSE)**
  - **Mean Absolute Error (MAE)**
  - **Root Mean Squared Error (RMSE)**
  - **R^2 Error**

Dr. Ritu Jain

Evaluation Metrics for Linear Regression

- **Mean Square Error (MSE)** is an evaluation metric that calculates the average of the squared differences between the actual and predicted values for all the data points. The difference is squared to ensure that negative and positive differences don't cancel each other out.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y_i})^2$$

Here,

- n is the number of data points.
- y_i is the actual or observed value for the ith data point.
- $\widehat{y_i}$ is the predicted value for the ith data point.

- MSE is a way to quantify the accuracy of a model's predictions.
- By squaring errors, it penalizes larger mistakes more strongly, making it sensitive to outliers.

Mean Absolute Error (MAE)

- **Mean Absolute Error (MAE)** is an evaluation metric used to calculate the accuracy of a regression model.
- MAE measures the average absolute difference between the predicted values and actual values.
- Mathematically MAE is expressed as:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|Y_i - \widehat{Y_i}|$$

Dr.

Here,

- n is the number of observations
- Y_i represents the actual values.
- $\widehat{Y_i}$ represents the predicted values

Root Mean Squared Error (RMSE)

- Root Mean Squared Error is the square root of MSE. It maintains the strong penalty for large errors while converting the result back to the original unit of the target variable, improving interpretability.
- In mathematical notation, it can be expressed as:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Where:

- n = number of observations
- y_i = actual value
- $\hat{y}_i$ = predicted value

- These metrics heavily penalize large errors

Coefficient of Determination (R-squared)

- R-squared represents the proportion of variance in the target variable that is explained by the regression model.
- It provides insight into how well the model captures underlying data patterns. Range: 0 to 1

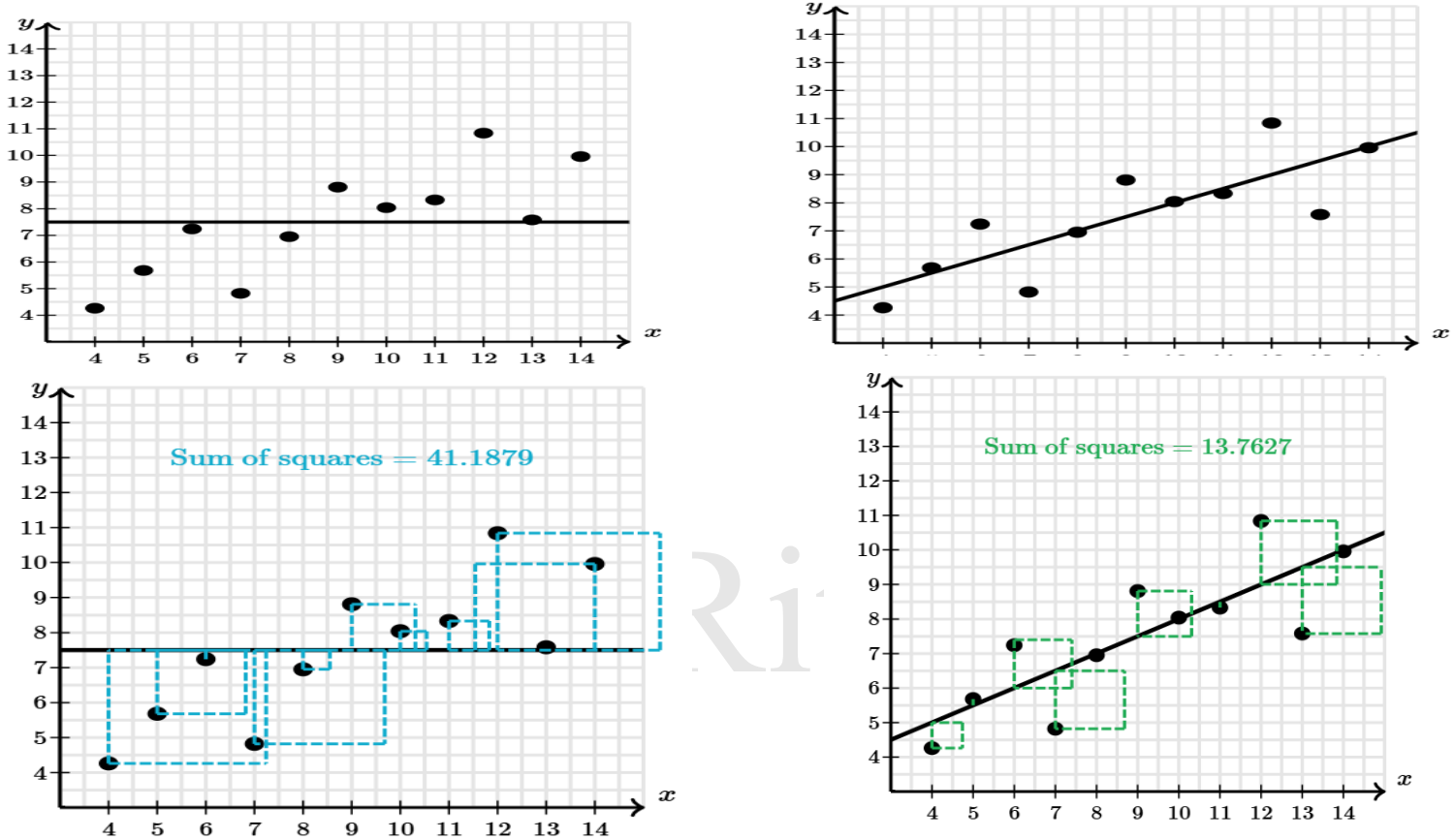
$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Where:

- $\bar{y}$ = mean of actual values
- Numerator = residual sum of squares
- Denominator = total sum of squares

- **Range of R^2**
  - **$R^2 = 1$** → Perfect fit (model explains 100% variance)
  - **$R^2 = 0$** → Model explains none of the variance
  - **$0 < R^2 < 1$** → Partial explanation of variance

Coefficient of Determination (R-squared)



We can represent this reduction as a percentage of the original amount of prediction error: (41.1879-13.7627)/41.1879= 0.6659

Prog: Simple Linear Regression along with metrics: MSE, MAE, RMSE, R Square

- Will be discussed in lab session

Dr. Ritu Jain

Model Evaluation Concepts

- **Generalization:** The model's ability to apply what it learned to unseen data.
- **Generalization Error:** The difference between the model's predictions and the actual outcomes on unseen data.
  - **In Regression:** Difference between predicted and actual continuous values.
  - **In Classification:** Proportion of incorrect predictions (e.g., misclassifications).
- **Objective:** Minimize generalization error to ensure the model performs well during **training, tuning, and testing** phases, not just on the training data.
- *A good ML model should fit the data well while generalizing effectively to new data.*

Dr. Ritu Jain

Model Evaluation Concepts

- **How to Measure Generalization Error:** The goal is to evaluate how well a model generalizes to **unseen data** by assessing its performance on different datasets. This connects directly to the **ultimate goal** of minimizing generalization error.
- **Training Dataset:** Used to **train** the model and learn parameters.
  - **Training Error:** Shows how well the model fits the training data.
- **Validation Dataset:** Used to **tune hyperparameters** and select the best-performing model.
  - **Validation Error:** Indicates how well the model can generalize during the development phase.
- **Test Dataset:** Used **only for final evaluation** after model selection.
  - **Test Error:** Provides an **unbiased estimate** of how the model performs on completely new, unseen data.

Dr. Ritu Jain

Empirical evaluation of Model

- Empirical evaluation relies on **specific performance metrics** (e.g., accuracy, precision, recall, RMSE) to quantify these errors.
- **Evaluation Metrics:**
- Focuses on **empirical, quantitative measurement** of model performance.
- Metrics include:
  - **Regression:** MSE, MAE, R^2
  - **Classification:** Accuracy, Precision, Recall, F1-score, ROC-AUC
- **Purpose:** Measures **how well a model actually performs** on training, validation, and test datasets.

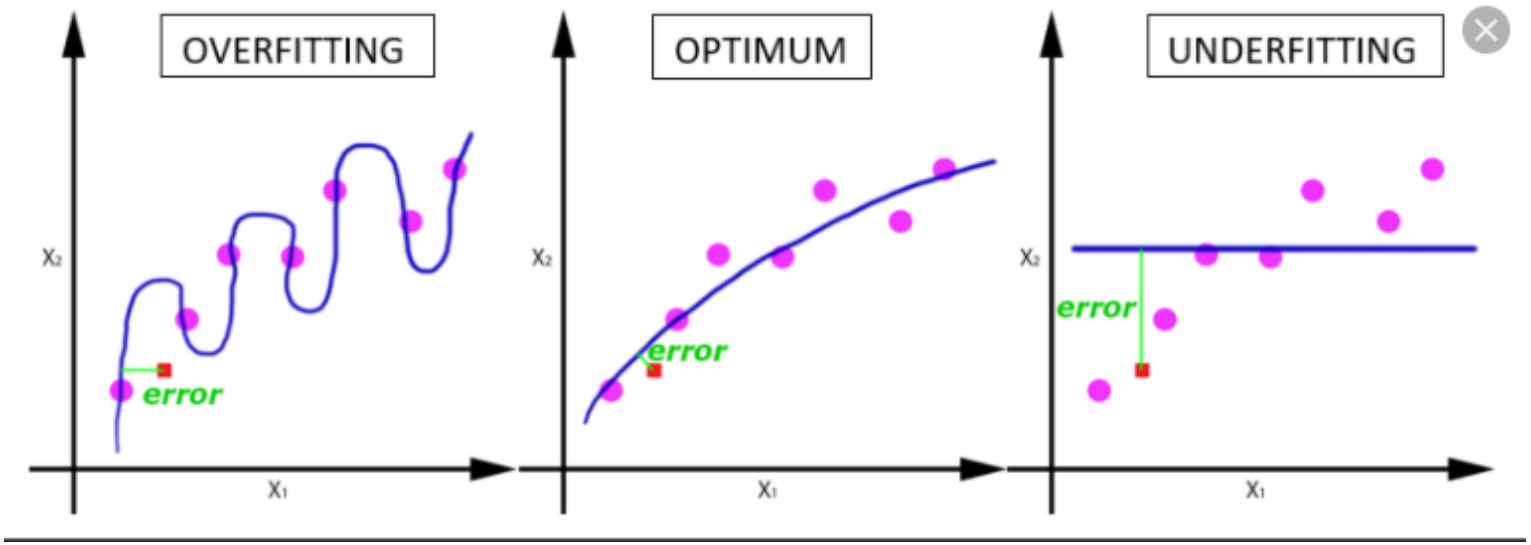
Dr. Ritu Jain

Underfitting and Overfitting in ML

- Machine learning models should learn useful patterns from training data. When a model learns too little or too much, we get **underfitting or overfitting**.
- **Underfitting** means that the model is *too simple* and *does not cover all real patterns in the data.*
- **Overfitting** means that the *model learns not just the underlying pattern, but also noise or random quirks in the training data. model memorizes training data*
- A good model finds the right *spot,* it is complex enough to capture real patterns, but not so complex that it "memorizes" noise

D Y PATIL UNIVERSITY
NAVI MUMBAI

Overfitting and Underfitting



Underfitting : Straight line trying to fit a curved dataset but cannot capture the data's patterns, leading to poor performance on both training and test sets.

Overfitting: A squiggly curve passing through all training points, failing to generalize performing well on training data but poorly on test data.

Appropriate Fitting: Curve that follows the data trend without overcomplicating to capture the true patterns in the data.

What is Overfitting?

- Overfitting occurs when a model learns both the underlying patterns and the noise in the training data to an extent that it negatively impacts the performance of the model on new data.
- An overfitted model has a low error on training data but performs poorly on unseen (test) data.
- Overfitting is often a result of an excessively complex model with too many parameters relative to the amount of training data.
- Causes of Overfitting:
  - Model too complex
  - Too many features
  - Inadequate training data
  - Lack of regularization (techniques used to prevent overfitting)
  - High variance
- ***Variance***: *Error that happens when a machine learning model learns too much from the data, including random noise.*

Dr. Ritu Jain

What is Underfitting?

- Underfitting occurs when a model is too simple, incapable of capturing the underlying pattern of the data, and performs poorly even on training data.
- An underfitted model has a high error on both the training and the test data.
- **Causes of Underfitting:**
  - Model is too simple with very few parameters
  - Excessive bias
  - Inadequate learning time during training, leading to poor optimization of model parameters
- In case of underfitting, Variance is low because the model gives similar outputs even if the data changes

$$Underfitting = High\ Bias + Low\ Variance$$

Dr. Ritu Jain

bias-variance tradeoff

- **Balancing Overfitting and Underfitting**
- *The goal in most machine learning tasks is to achieve a good bias-variance tradeoff.*
- **Bias** is an error introduced in the model due to oversimplification of the machine learning algorithm.
  - ***Bias*** *is like assuming all birds can only be small and fly, so the model fails to recognize big birds like ostriches or penguins that can't fly and get biased with predictions.*
- **Variance** is an error introduced in the model due to excessive complexity that leads it to learn noise from the training data as well.

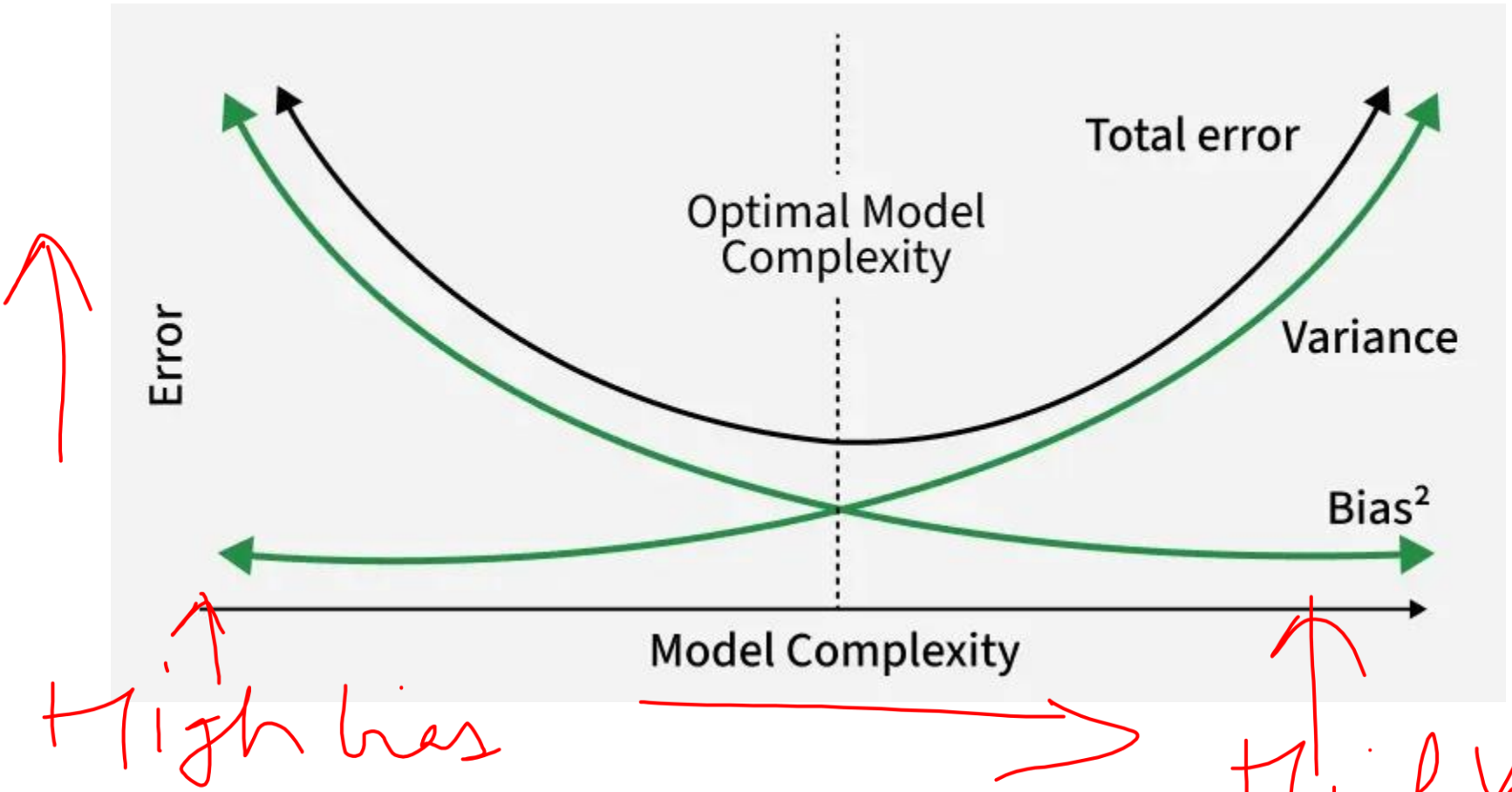
Dr Ritu Jain

bias-variance tradeoff

- The relationship between bias and variance is often referred to as the bias-variance tradeoff, which highlights the need for balance:
  - Increasing model complexity reduces bias but increases variance (risk of overfitting).
  - Simplifying the model reduces variance but increases bias (risk of underfitting).
- The goal is to find an optimal balance where both bias and variance are minimized, resulting in good generalization performance.

Dr. Ritu Jain

bias-variance tradeoff



How to Address Overfitting and Underfitting?

- **Techniques to Reduce Underfitting**
  - Use a more complex model
  - Add new features and perform feature engineering
  - Reduce regularization
  - Train for more epochs
  - Scale features properly
- **Techniques to Reduce Overfitting**
  - Collect more training data
  - Reduce model complexity
  - Use regularization (L1/L2)
  - Apply dropout (for neural networks)
  - Use early stopping
  - Clean noisy data
-

Dr. Ritu Jain

Regularization in Machine Learning

- Regularization is a technique used in machine learning to **prevent overfitting.**
- *It adds a penalty term to the loss function, encouraging simpler models that generalize better to unseen data.*
- *It restricts model complexity by shrinking coefficient estimates (weights) toward zero, balancing the trade-off between bias and variance.*

Dr. Ritu Jain

How to Address Overfitting

- Earlier we just plotted a higher order function - saw that it looks "too curvy"
  - If you have lots of features and little data - overfitting can be a problem
- How do we deal with this?
  - 1) **Reduce number of features**
  - 2) **Regularization**
    - Keep all features, but reduce magnitude of parameters θ
    - Works well when we have a lot of features, each of which contributes a bit to predicting y

Dr. Ritu Jain

Regularization

- Regularization
  - Small values for parameters corresponds to a simpler hypothesis
  - A simpler hypothesis is less prone to overfitting
- Another example
  - Have 100 features $x_1, x_2, \ldots, x_{100}$
  - Unlike the polynomial example, we don't know what are the high order terms
    - How do we pick the ones to pick to shrink?
  - With regularization, take cost function and modify it to shrink all the parameters
    - Add a term at the end
      - This regularization term shrinks every parameter
      - By convention you don't penalize θ_0 - minimization is from θ_1 onwards

Dr. Ritu Jain

Cost function optimization for L2 regularization

- **Generalized Cost Function:**
  - The λ, or lambda, is the **regularization parameter**. It determines how much the costs of our theta parameters are inflated.

$$min_\theta \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^{n} \theta_j^2$$

Dr. Ritu Jain

  - Using the above cost function with the extra summation, we can smooth the output of our hypothesis function to reduce overfitting.
- In practice, if you include θ_0 has little impact
- **λ** is the **regularization parameter**
  - Controls a trade off between our two goals
    - Want to fit the training set well
    - Want to keep parameters small

Cost function optimization for regularization

- If lambda is chosen to be too large
- What would happen if λ=0 or is too small ?

Dr. Ritu Jain

Gradient Descent for L2 regularization

- We will modify our gradient descent function to separate out θ_0 from the rest of the

Repeat {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \left[\left(\frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{m} \theta_j \right] \qquad j \in \{1, 2 \ldots n\}$$

}

- The term $(\lambda/m)\theta_j$ performs our regularization. With some manipulation our update rule can also be represented as:

$$\theta_j := \theta_j (1 - \alpha \frac{\lambda}{m}) - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- The first term in the above equation, $1 - \alpha(\lambda/m)$ will always be less than 1. Intuitively you can see it as reducing the value of θj by some amount on every update.
- Notice that the second term is now exactly the same as it was before.

Prof. Kritu Jain

Types of Regularization

- Types of regularization techniques:
  1. **L1 Regularization**
  2. **L2 Regularization**
- Each regularization techniques apply penalties in different ways to control model complexity and improve generalization.

Dr. Rijul Jain

L2 Regularization

- **Mechanism:** It adds a penalty proportional to the sum of the squared magnitudes of the weights to the cost function.

$$\textbf{Formula: } J(\theta) = \text{OriginalCost} + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$$

m : no. of instances
n : no of features

- **Effect:** It forces the weights to be small (close to zero) but not exactly zero.
  - This leads to a smoother model that is less sensitive to small changes in input features.
- **Key Insight:** Often referred to as "weight decay" because during gradient descent, it reduces the weight magnitude in each iteration.
- **Ridge regression** is a linear modeling technique that prevents overfitting by adding an L2 penalty (proportional to the square of coefficient magnitudes) to the cost function.

Dr. Ritu Jain

L1 Regularization

- **Mechanism:** It adds a penalty proportional to the **sum of the absolute values** of the weights (θ_j or W) to the cost function.

- **Formula:** $J(\theta) = \text{OriginalCost} + \frac{\lambda}{2m} \sum_{j=1}^{n} |\theta_j|$

- **Effect:** It forces many parameter weights to become **exactly zero**.

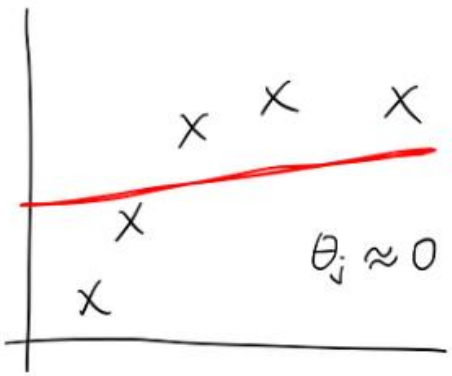
- **Use Case:** Highly effective for feature selection, as it acts as a method to ignore irrelevant features in a model.

Lasso (Least Absolute Shrinkage and Selection Operator) regression is a linear modeling technique that uses L1 regularization to improve model generalization and prevent overfitting by adding a penalty proportional to the sum of absolute values of coefficients.

Regularization

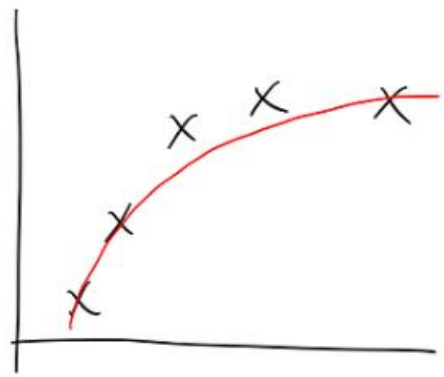
if λ is large:

θ_j will decrease automatically



$\theta_0 + \cancel{\theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4}$

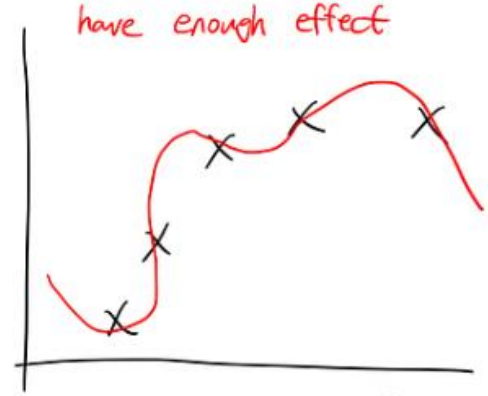
$\boxed{\text{underfit}}$



if λ is small:

ex) $\lambda = 0$

regularization term will not have enough effect



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

$\boxed{\text{overfit}}$

Regularization

- **λ (Lambda):** The regularization parameter λ controls the trade-off between fitting the training data well and keeping the model simple (low weights).
- **λ too high:** Leads to underfitting (high bias).
- **λ too low:** Leads to continued overfitting (high variance).
- **Bias Term:** It is common practice **not** to regularize the bias term (θ_0 or b), as doing so makes little difference in practice.

Regularization

Type	Penalty	Effect on Weights	Feature Selection?
L2	$\lambda \sum w^2$	Small, non-zero values	No
L1	$\lambda \sum \lvert w \rvert$	Many become zero	Yes (Sparse)

w. is same as 0

When to use L1 Regularization

Use L1 Regularization When:

- You need **Feature Selection**: L1 is ideal when you suspect only a small subset of your features are actually useful. It drives the weights of irrelevant features to exactly zero, effectively removing them from the model.
- You want a **Sparse Model:** Because it eliminates features, L1 produces simpler, more interpretable models. This is particularly useful in high-dimensional datasets where feature redundancy is common.
- You have **High-Dimensional Data**: L1 is often preferred when the number of features is very large relative to the number of training examples

Dr. Ritu Jain

When to use L2 Regularization

Use L2 Regularization When:

- You want **Balanced Weights:** L2 shrinks weights toward zero but never exactly to zero. It is the "default" choice in many deep learning contexts (often called weight decay) because it keeps all features in the model but limits their individual influence.
- You have **Correlated Features:** L2 handles multicollinearity better than L1. While L1 might randomly pick one of two highly correlated features and drop the other, L2 will distribute the weights more evenly across both.
- You **prioritize Accuracy/Stability:** In practice, L2 often yields better predictive performance and more stable models because it utilizes all available information rather than discarding it

Dr. Ritu Jain